

Titanic Data Science — Cleaning, EDA & ML Guide

Purpose: explain not only **what** operation is performed, but also **why** it is performed, **when** it should be used, and **what would change** if the data had a different problem.

Dataset: Kaggle — Titanic: Machine Learning from Disaster. The official Kaggle page describes train.csv as 891 passengers with the Survived ground-truth target.

Kaggle link: <https://www.kaggle.com/competitions/titanic/data?select=train.csv>

Provided files: titanic_train.csv, Cleaning_Titanic_Complete_DataScience.ipynb, this PDF guide.

1. The Data Science Mindset

Do not ask: "Which pandas operation should I use?"

Ask: "What problem does the data have, and what operation solves that problem without damaging useful information?"

Example: if a column has missing values, first measure the amount and meaning of missingness. Only then decide between dropping rows, imputing values, adding a missing category, or dropping the column.

2. Load data — pd.read_csv()

WHAT: converts CSV text into a pandas DataFrame.

WHY: DataFrame operations make tabular data inspectable and transformable.

IF / WHEN: If the file path is wrong: fix the path. If encoding is unusual: specify encoding. If separators differ: specify sep.

CAUTION: Never assume the file loaded correctly; immediately check shape, columns and dtypes.

Example:

```
df = pd.read_csv("titanic_train.csv")
```

3. shape, columns, head(), tail(), sample()

WHAT: shape gives rows/columns; columns gives names; head/tail show edges; sample shows random observations.

WHY: these are fast sanity checks. sample() is useful because the first rows may not represent the full dataset.

IF / WHEN: If data is time ordered, sample helps inspect random coverage; if order itself matters, preserve it for time-series tasks.

CAUTION: Do not use only head() to understand a dataset.

Example:

```
df.shape\n df.head()\n df.sample(5, random_state=42)
```

4. info() and dtypes

WHAT: shows column types and non-null counts.

WHY: type errors cause wrong calculations and model failures. Missingness is also visible here.

IF / WHEN: If a number is stored as object/string, convert it after checking whether the strings are valid numeric values.

CAUTION: Do not blindly convert every object column to numeric; many categorical columns are supposed to be text.

Example:

```
df.info()\n df.dtypes
```

5. describe()

WHAT: summarizes numeric distributions using count, mean, std, quartiles, min and max.

WHY: detects suspicious ranges, scale differences and possible outliers.

IF / WHEN: If the mean is far from the median, investigate skew/outliers. For categories, use describe(include='object') or value_counts().

CAUTION: Summary statistics are evidence, not proof of an error.

Example:
`df.describe().T`

6. Missing values — `isna().sum()`

WHAT: counts missing observations.

WHY: most ML algorithms cannot directly use NaN values, and careless deletion can reduce sample size or bias results.

IF / WHEN: Small missing fraction → deletion may be acceptable; numeric feature → median/mean/model-based imputation may work; categorical → mode or explicit 'Unknown'; very high missingness → consider dropping or extracting a useful indicator.

CAUTION: For Titanic, Age has 177 missing values, Cabin has 687, and Embarked has 2. Cabin's missingness is so high that raw Cabin is not kept; a CabinDeck feature is extracted first.

Example:
`df.isna().sum()`

7. Duplicate rows — `duplicated()/drop_duplicates()`

WHAT: finds exact repeated rows.

WHY: accidental duplicates can give some observations extra weight.

IF / WHEN: If duplicates represent legitimate repeated events, keep them. If PassengerId must be unique, also check that key.

CAUTION: Never drop duplicates without understanding what a row represents.

Example:
`df.duplicated().sum()\nndf = df.drop_duplicates()`

8. `loc`, `iloc`, filtering

WHAT: `loc` is label-based; `iloc` is position-based; boolean masks filter rows.

WHY: row/column selection is fundamental to EDA and cleaning.

IF / WHEN: Multiple conditions use `&` and `|` with parentheses.

CAUTION: Using `and/or` instead of `&/|` with pandas Series commonly causes errors.

Example:
`df.loc[df["Age"] > 60, ["Name", "Age"]]`

9. Sorting

WHAT: `sort_values` orders records.

WHY: helps inspect highest/lowest values, investigate outliers and rank observations.

IF / WHEN: If you need deployment/reuse, save the cleaned/model-ready data and the fitted pipeline.

CAUTION: Sort by multiple columns when one column is not enough to define the order.

Example:
`df.sort_values("Fare", ascending=False).head()`

10. `value_counts()`, `nunique()`

WHAT: count categories and distinct values.

WHY: detects unexpected labels, category size, high cardinality and class imbalance.

IF / WHEN: If a category appears only once, decide whether it should be grouped into a rare category before one-hot encoding.

CAUTION: Do not group rare categories without considering whether they carry important signal.

Example:
`df["Sex"].value_counts(dropna=False)`

11. groupby(), agg(), pivot_table(), crosstab()

WHAT: summarize data by groups.

WHY: converts raw rows into business/statistical questions such as survival rate by sex or class.

IF / WHEN: Use mean for rates when target is 0/1; count for volume; median for skewed numeric variables such as fare.

CAUTION: A group summary describes association; it does not prove causation.

Example:
`df.groupby("Sex")["Survived"].mean()`

12. Data-quality rules

WHAT: explicit checks for impossible or invalid values.

WHY: cleaning should correct real data-quality problems, not simply make distributions look nicer.

IF / WHEN: For Titanic: Age < 0 is impossible, Fare < 0 is invalid, Survived must be 0/1, Pclass must be 1/2/3.

CAUTION: If a value is unusual but possible, investigate instead of deleting it.

Example:
`((df["Age"] < 0).sum())`

13. String cleaning and feature extraction

WHAT: strip spaces, standardize text and extract structured information from strings.

WHY: raw text often contains hidden variables. Titanic Name contains titles such as Mr, Mrs, Miss and Master.

IF / WHEN: If text has inconsistent capitalization/spaces, normalize first. If a text field is high-cardinality, extract useful components rather than one-hot encoding every unique value.

CAUTION: Text cleaning must preserve meaningful distinctions.

Example:
`df["Name"] = df["Name"].str.strip()`

14. Feature engineering

WHAT: create new variables from existing variables.

WHY: a useful feature can represent a pattern more directly than raw columns.

IF / WHEN: Titanic examples: FamilySize = SibSp + Parch + 1; IsAlone; FarePerPerson; CabinDeck; Title.

CAUTION: Feature engineering should be based on domain logic or exploratory evidence. Avoid features that use future/target information.

Example:
`df["FamilySize"] = df["SibSp"] + df["Parch"] + 1`

15. Missing-value imputation

WHAT: replace missing values with a justified estimate/category.

WHY: keeps useful rows and allows algorithms to operate.

IF / WHEN: Age is imputed using group medians (Sex × Pclass × Title) with a global-median fallback. Embarked uses the mode. Fare uses the median.

CAUTION: For final ML, fit imputers only on training data. The pipeline in the notebook demonstrates leakage-safe preprocessing.

Example:
`df["Age"] = df["Age"].fillna(df["Age"].median())`

16. Dropping a column

WHAT: removes a feature.

WHY: a feature may be unusable, redundant, too noisy, mostly missing, an identifier, or a leakage source.

IF / WHEN: Cabin has 687 missing values out of 891. Instead of throwing away all cabin information, the notebook extracts CabinDeck and then drops raw Cabin.

CAUTION: Dropping is a modeling decision, not a universal cleaning rule.

Example:
`df_clean = df.drop(columns=["Cabin"])`

17. Outliers — IQR

WHAT: identifies values outside $Q1 - 1.5 \times IQR$ and $Q3 + 1.5 \times IQR$.

WHY: extreme values can affect some algorithms and statistics.

IF / WHEN: First investigate whether an outlier is a real observation. Then choose keep, transform, cap or remove.

CAUTION: Titanic's high fares may be legitimate, so they should not automatically be deleted.

Example:
`IQR = Q3 - Q1
nupper = Q3 + 1.5 * IQR`

18. Visualization

WHAT: plots reveal distributions and relationships.

WHY: visual evidence is often easier to interpret than tables alone.

IF / WHEN: Bar charts help categories; histograms/boxplots help distributions; scatterplots help numeric relationships.

CAUTION: Visualization does not prove causality and should be combined with statistical/model evidence.

Example:
`df.boxplot(column="Fare", by="Pclass")`

19. Target and feature separation

WHAT: split the target y from predictors X.

WHY: the target is what the model is trying to predict and must not accidentally enter the feature matrix.

IF / WHEN: Also remove identifiers and leakage-prone fields.

CAUTION: PassengerId is an identifier; Survived is the target. Name/Ticket are high-cardinality raw fields handled or removed in the notebook.

Example:
`X = df.drop(columns=["Survived"])\ny = df["Survived"]`

20. Train/test split

WHAT: reserve unseen observations for evaluation.

WHY: training performance is not a reliable measure of generalization.

IF / WHEN: Use stratify for classification when class proportions matter. For time series, use time-aware splitting instead of random splitting.

CAUTION: Never tune decisions repeatedly on the final test set.

Example:
`train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)`

21. Encoding categorical variables

WHAT: converts categories into numeric model inputs; one-hot encoding creates indicator columns.

WHY: most ML estimators need numeric matrices.

IF / WHEN: One-hot is usually safe for nominal categories. Ordinal encoding is appropriate only when order has meaning.

CAUTION: Use `handle_unknown='ignore'` so new categories do not crash inference.

Example:
`OneHotEncoder(handle_unknown="ignore")`

22. Scaling

WHAT: transforms numeric variables to comparable scales.

WHY: distance/gradient-based models such as KNN and Logistic Regression can be sensitive to feature scale.

IF / WHEN: Tree-based models generally do not require scaling.

CAUTION: Scaling must be learned from training data, not the whole dataset.

Example:
`StandardScaler()`

23. Pipeline and ColumnTransformer

WHAT: chains preprocessing and modeling into one reproducible object.

WHY: prevents inconsistent transformations and reduces data leakage.

IF / WHEN: Use separate numeric/categorical preprocessing because their cleaning rules differ.

CAUTION: A pipeline is especially important when deploying the model to new data.

Example:
`Pipeline([("preprocessor", preprocessor), ("model", LogisticRegression())])`

24. Model comparison

WHAT: train multiple algorithms on the same train/test framework.

WHY: no single algorithm is best for every dataset.

IF / WHEN: The notebook compares Logistic Regression, Decision Tree, Random Forest and KNN.

CAUTION: Compare using metrics relevant to the problem, not only accuracy.

Example:
`models = {"Logistic Regression": LogisticRegression(), ...}`

25. Accuracy, Precision, Recall, F1

WHAT: different ways to evaluate classification.

WHY: each metric answers a different question.

IF / WHEN: Accuracy = overall correctness. Precision = how many predicted positives were correct. Recall = how many actual positives were found. F1 balances precision and recall.

CAUTION: If false negatives are costly, recall may matter more than accuracy.

Example:
`classification_report(y_test, pred)`

26. Confusion matrix

WHAT: table of TN, FP, FN, TP.

WHY: shows exactly what types of errors the classifier makes.

IF / WHEN: Use it when business decisions depend on the cost of different error types.

CAUTION: Do not interpret only the diagonal; inspect false positives and false negatives.

Example:
`confusion_matrix(y_test, pred)`

27. ROC-AUC

WHAT: summarizes ranking quality across classification thresholds.

WHY: useful when the default 0.5 threshold is not the only possible operating point.

IF / WHEN: For strongly imbalanced problems, also inspect Precision-Recall curves and class-specific metrics.

CAUTION: AUC is not the same as accuracy and does not tell you the best threshold by itself.

Example:
`roc_auc_score(y_test, proba)`

28. Cross-validation

WHAT: repeatedly trains/evaluates across folds.

WHY: one split can be lucky. Cross-validation provides a more stable estimate.

IF / WHEN: Use StratifiedKFold for classification.

CAUTION: Cross-validation should still respect the data-generation process; time series needs time-aware validation.

Example:
`cross_val_score(pipe, X, y, cv=cv, scoring="accuracy")`

29. Model interpretation

WHAT: inspect coefficients or feature importance.

WHY: helps understand what the model uses and supports debugging/communication.

IF / WHEN: Logistic coefficients show direction on the transformed feature scale. Tree importance shows model usage, not causality.

CAUTION: Do not say “feature X causes survival” merely because its importance is high.

Example:
`coef_df.sort_values("abs_coefficient", ascending=False)`

30. Save outputs and reproducibility

WHAT: save cleaned data, model-ready data and the fitted pipeline.

WHY: a Data Scientist should be able to reproduce and reuse the workflow.

IF / WHEN: In production, also save dependency versions, configuration, data version and model metadata.

CAUTION: Never rely on an interactive notebook state as the only copy of a production pipeline.

Example:
`df_clean.to_csv("titanic_cleaned.csv", index=False)`

Decision Tree — “If this, then that”

Use this mental model whenever you receive a new dataset:

Problem found	Question to ask	Possible action
Missing values	How much is missing? Does missingness have meaning?	Drop / impute / Unknown / missing indicator / drop column
Duplicates	Are repeated rows real observations?	Keep or drop exact duplicates
Wrong dtype	What does the value actually represent?	Convert to numeric/date/category/text
Invalid values	Impossible or merely unusual?	Correct using source rules or investigate
Outliers	Error or legitimate extreme observation?	Keep / transform / cap / remove with justification
High-cardinality text	Is there hidden structured information?	Extract title/group or use text methods
Categorical data	Nominal or ordered?	One-hot or appropriate ordinal representation
Numeric scale	Does the model depend on distance/gradient?	Scale for KNN/logistic; usually unnecessary for trees
Leakage risk	Was information learned from test/future/target?	Remove feature or fit transformation only on train
Class imbalance	Does accuracy hide minority-class errors?	Use stratification and class-aware metrics
Model performance	Is one split reliable?	Cross-validation + final untouched test set

Final Data Scientist Checklist

Before saying “my data is clean,” verify all of the following:

1. Row meaning is understood.
2. Target is clearly defined.
3. Shape and schema are checked.
4. Missingness is quantified and justified.
5. Duplicate logic is understood.
6. Data types are correct.
7. Invalid values are checked.
8. Categorical levels are inspected.
9. Outliers are investigated rather than blindly deleted.
10. Features are engineered from valid information.
11. Leakage is prevented.
12. Preprocessing is reproducible in a pipeline.
13. Train/test split matches the problem.
14. Multiple models are compared fairly.
15. Metrics match the decision problem.
16. Cross-validation is used where appropriate.
17. The final model can process new data using the exact same transformations.
18. The cleaning decisions are documented so another person can reproduce them.

What changes if the dataset changes?

The operations are not a fixed recipe. If a new dataset has **no missing values**, you do not need imputation. If it has **dates**, extract date parts and use appropriate time-aware validation. If it is **time series**, random train/test splitting is usually inappropriate. If it is **text**, TF-IDF/embeddings may replace ordinary one-hot encoding. If it is **highly imbalanced**, accuracy may become misleading. If it contains **target leakage**, the first priority is to remove the leakage.

Key principle: operation → reason → evidence → validation. That is the habit that separates routine pandas usage from Data Science.